EXPERIMENT 9

Aim: Implement Recurrent Neural Network (RNN) / LSTM for time series or text data

1. Dataset Source

Dataset used: **Air Quality Dataset (UCI Repository)**

Link: https://archive.ics.uci.edu/dataset/360/air+quality

2. Dataset Description

The Air Quality dataset contains environmental sensor data collected in an Italian city.

- **Type:** Time Series Data
- **Instances:** ~9,000 records
- **Attributes:** 15 features

Key Features:

- CO(GT): Carbon monoxide concentration (target variable used)
- PT08.S1(CO): Sensor response
- NMHC(GT): Non-methane hydrocarbons
- NOx(GT): Nitrogen oxides
- Temperature (T)
- Relative Humidity (RH)
- Absolute Humidity (AH)

Target Variable

- The experiment uses **CO(GT)** (converted to numeric and cleaned)

Data Characteristics

- Contains missing values represented as **-200**
- Requires preprocessing (cleaning + normalization)
- Sequential/time-dependent → suitable for LSTM

3. **Mathematical Formulation of LSTM**

4. **Algorithm Limitations**

- Requires **large datasets** for good performance
- Computationally expensive compared to traditional models
- Sensitive to hyperparameters (epochs, learning rate, units)
- Cannot handle **very noisy data** effectively
- Training can be slow on CPU (as seen in your output)
- Risk of overfitting if not properly tuned

5. **Methodology / Workflow**

Step-by-Step Process

1. **Data Loading**
   - Load CSV file using `readtable()`
2. **Data Cleaning**
   - Convert column to numeric
   - Replace **-200 with NaN**
   - Remove missing values

3. **Normalization**
    - Normalize data for better training
4. **Sequence Creation**
    - Input: XtX_tXt
    - Output: Xt+1X_{t+1}Xt+1
5. **Model Design**
    - Sequence Input Layer
    - LSTM Layer (50 units)
    - Fully Connected Layer
    - Regression Layer
6. **Training**
    - Optimizer: Adam
    - Epochs: 50
7. **Prediction**
    - Predict values using trained model
8. **Visualization**
    - Plot Actual vs Predicted values

6. **Performance Analysis**

- Training completed for **50 epochs**
- RMSE values fluctuate between **~0.5 to 1.3** (page 3–4 logs)
- Loss reduces gradually → model is learning

Graph Observation:

- Blue = Actual values
- Red = Predicted values
- Predictions follow trend but have slight deviations

Conclusion:

- Model captures **time-series pattern successfully** ● Some error exists due to: ○ Noise in data
    - Simple model architecture

7. **Hyperparameter Tuning Parameters Used:**

- LSTM Units = 50
- Epochs = 50
- Optimizer = Adam
- Learning Rate ≈ 0.001

Tuning Strategies

Parameter	Effect
Increase LSTM units	Better learning but more computation
Increase epochs	Improves accuracy but risk of overfitting
Learning rate	Too high → unstable, too low → slow learning
Batch size	Affects convergence speed

Possible Improvements

- Increase LSTM units (e.g., 100)
- Add Dropout layer to prevent overfitting
- Use multiple LSTM layers (Deep LSTM)

- Try Bidirectional LSTM

8. Code and Output

```
% Load dataset data =
readtable('AirQualityUCI.csv');


% Convert column to numeric (IMPORTANT FIX) series
= str2double(data{:,3});


% Replace invalid values (-200)
series(series == -200) = NaN;


% Remove missing values series
= series(~isnan(series));


% Normalize series =
normalize(series);


% Create sequences
  X = series(1:end-1);
  Y = series(2:end);


XTrain = num2cell(X');
YTrain = num2cell(Y');


% LSTM network
layers = [
   sequenceInputLayer(1) lstmLayer(50)
   fullyConnectedLayer(1)
   regressionLayer
];


% Training options options =
trainingOptions('adam',          ...
'MaxEpochs', 50, ...
   'Plots','training-progress');
% Train
```

```
net = trainNetwork(XTrain, YTrain, layers, options);

% Predict
YPred = predict(net, XTrain);

YPred = cell2mat(YPred);
YTrue = cell2mat(YTrain);

% Plot results figure; plot(YTrue,
'b'); hold on; plot(YPred, 'r');
legend('Actual','Predicted');
title('LSTM Time Series Prediction');
```

9. Conclusion

The LSTM model successfully performs time-series forecasting on air quality data. It captures temporal dependencies effectively, though performance can be further improved with better tuning and model complexity.